

A Complete Navigation Stack for Micromouse-Scale Maze Solving: From Grid Representation to Clothoid-Based Trajectory Execution

Tomás S. Giomi

Department of Robotics and Control

Instituto La Salle Florida

Email: tomasgiomi@gmail.com, tgiomi@lasalleflorida.edu.ar

Abstract—This paper presents a comprehensive, mathematically rigorous navigation architecture for autonomous maze-solving robots operating in structured environments. The system integrates six primary components: a finite grid graph representation with binary wall belief states, a Dijkstra-based flood-fill heuristic field, A* path planning with guaranteed optimality, a Trémaux-style explorer ensuring complete coverage, clothoid-based trajectory generation providing C^2 continuity, and a Kalman filter for robust state estimation. Each component is analyzed for its mathematical properties, computational complexity, and the invariants that guarantee system-wide correctness. The architecture achieves provable completeness in exploration, optimality in path planning, and physical feasibility in trajectory execution, making it suitable for real-time embedded implementation.

Index Terms—A* search, admissible heuristics, clothoid curves, Dijkstra’s algorithm, Kalman filtering, maze solving, path planning, Trémaux algorithm

I. INTRODUCTION

Autonomous maze solving represents a canonical challenge in robotics that demands the integration of mapping, planning, and control in a resource-constrained setting. The problem requires a robot to explore an unknown environment, construct an internal representation, plan optimal paths through that representation, and execute those paths with physically realizable trajectories.

The Micromouse competition exemplifies these challenges: a small autonomous robot must navigate a 16×16 cell maze, where each cell measures 0.18 m per side. The robot must first explore to discover the maze layout (the search phase), then navigate from a corner start to the center goal along the shortest possible path (the speed phase). This dual requirement—exploration then exploitation—mirrors the fundamental exploration-exploitation dilemma in autonomous systems.

We present a navigation architecture that formalizes each subsystem as a well-defined mathematical structure with proven properties. The key contributions of this work are:

- 1) A formal specification of the grid world model, including coordinate transformations, move sets, and the critical distinction between pessimistic and optimistic wall belief states that enable dual-purpose map usage.

- 2) A proof that the flood-fill heuristic field is both admissible and consistent, guaranteeing A* optimality.
- 3) A Trémaux-based exploration algorithm augmented with information-theoretic scoring that guarantees complete coverage while minimizing exploration time.
- 4) A clothoid-based trajectory generator achieving C^2 continuity, coupled with jerk-limited velocity profiling that respects centripetal and friction-circle constraints.
- 5) A Kalman filter fusing wheel odometry, gyroscopic heading, and wall-proximity measurements to maintain consistent state estimation.

The remainder of this paper is organized as follows. Section II presents the grid world model and wall belief representation. Section III describes the flood-fill distance field computation. Section IV analyzes the A* planning algorithm and its optimality guarantees. Section V details the Trémaux explorer and its coverage properties. Section VI presents the clothoid generation and velocity profiling. Section VII describes the Kalman filter implementation. Section VIII summarizes the computational complexity across all components, and Section IX concludes with a discussion of the system invariants.

II. GRID WORLD MODEL

A. Cell Representation and Indexing

The maze environment is modeled as a finite grid graph $\mathcal{G} = (\mathcal{C}, \mathcal{E})$, where vertices correspond to cells and edges correspond to traversable connections between adjacent cells. The cell set is defined as:

$$\mathcal{C} = \{(x, y) \in \mathbb{Z}^2 : 0 \leq x < N, 0 \leq y < N\} \quad (1)$$

with $N = 16$, yielding 256 discrete states. Each cell is uniquely identified by its integer coordinates (x, y) . For efficient memory access and cache performance, we employ row-major linear indexing:

$$\text{idx}(x, y) = y \cdot N + x \in [0, N^2 - 1] \quad (2)$$

This transformation maps the 2D coordinate space to a contiguous 1D array, enabling $O(1)$ cell lookup and improving spatial locality for pathfinding algorithms that frequently traverse neighboring cells.

B. Eight-Directional Movement and Cost

The robot supports movement in eight directions, represented by the displacement set:

$$\mathcal{D} = \{(0, -1), (1, -1), (1, 0), (1, 1), (0, 1), (-1, 1), (-1, 0), (-1, -1)\} \quad (3)$$

corresponding to the cardinal directions North, Northeast, East, Southeast, South, Southwest, West, and Northwest, respectively. Each direction $d \in \mathcal{D}$ has an associated movement cost based on the Euclidean distance between cell centers:

$$\text{cost}(d) = \begin{cases} 1 & \text{if } d \in \{N, E, S, W\} \\ \sqrt{2} \approx 1.414 & \text{if } d \in \{NE, SE, SW, NW\} \end{cases} \quad (4)$$

This cost function accurately reflects the geometric distance traveled, which is essential for the admissibility of the heuristic function used in A* search. Without this differentiation, diagonal paths would be systematically undervalued, potentially leading to suboptimal trajectories.

C. Coordinate Frame Transformation

A critical detail in practical implementation is the inversion of the grid Y-axis when mapping to world coordinates. In the grid representation, the Y index increases southward (consistent with image and matrix conventions). However, in the physical world frame, Y typically increases northward. The transformation is:

$$y_{\text{world}} = -y_{\text{grid}} \cdot c \quad (5)$$

where $c = 0.18$ m is the physical cell dimension. The X-axis maintains its orientation in both frames. This transformation ensures that planned paths are correctly executed in the physical environment.

D. Wall Model and Belief States

Each cell maintains a binary belief state over its four cardinal walls. For each cell (x, y) and wall direction $w \in \{N=0, E=1, S=2, W=3\}$, we define two quantities:

$$W(x, y, w) \in \{0, 1\}, \quad K(x, y, w) \in \{0, 1\} \quad (6)$$

where W represents the actual wall state (0 = open, 1 = wall) and K represents whether the state is known (0 = unknown, 1 = known). This separation between ground truth and robot belief is fundamental to enabling dual-purpose map usage.

Wall Symmetry Constraint: Physical walls are shared between adjacent cells. This is enforced through the symmetry constraint:

$$W(x, y, w) := b \implies W(x + \Delta_x(w), y + \Delta_y(w), \text{opp}(w)) := b \\ K(\cdot) := 1 \text{ for both sides} \quad (7)$$

where $\text{opp}(w) = (w + 2) \bmod 4$ is the opposite direction, and Δ_{wall} maps directions to displacement vectors: N : (0, -1), E : (1, 0), S : (0, 1), W : (-1, 0).

E. Pessimistic and Optimistic Passability

The distinction between known and unknown walls enables two passability functions that serve different roles in the system:

Pessimistic Passability (used during safe navigation):

$$\text{pass}_{\text{pess}}(x, y, d) = \prod_{w \in \text{walls}(d)} [K(x, y, w) = 1 \wedge W(x, y, w) = 0] \\ \wedge [(x + \Delta_x(d), y + \Delta_y(d)) \in \mathcal{C}] \quad (8)$$

This function treats unknown walls as blocked. A move is passable only if all required walls are known and open. This guarantees safety during high-speed runs by preventing the robot from traversing unverified corridors.

Optimistic Passability (used during exploration):

$$\text{pass}_{\text{opt}}(x, y, d) = \prod_{w \in \text{walls}(d)} [K(x, y, w) = 0 \vee W(x, y, w) = 0] \\ \wedge [(x + \Delta_x(d), y + \Delta_y(d)) \in \mathcal{C}] \quad (9)$$

This function treats unknown walls as open, encouraging the explorer to investigate uncharted territory. The difference is in the logical condition: optimistic requires either unknown OR known-open, while pessimistic requires known AND open.

F. Diagonal Wall Clearance

For diagonal moves, additional wall clearance constraints apply to prevent the robot from passing through physically impossible gaps. The wall requirements for each diagonal direction are:

$$\begin{aligned} \text{walls}(NE) &= \{N, E\} \\ \text{walls}(SE) &= \{E, S\} \\ \text{walls}(SW) &= \{S, W\} \\ \text{walls}(NW) &= \{N, W\} \end{aligned} \quad (10)$$

For cardinal moves, only the wall in the direction of travel is required: $\text{walls}(d) = \{d\}$ for $d \in \{N, E, S, W\}$.

III. FLOOD-FILL DISTANCE FIELD

A. Dijkstra-Based Heuristic Computation

The distance field $f : \mathcal{C} \rightarrow \mathbb{R}_+$ maps each cell to its minimum path cost to the goal region. This field serves as the admissible heuristic for A* planning and as the guiding potential for the Trémaux explorer.

The field is computed via Dijkstra's shortest-path algorithm, initialized with all goal cells:

$$f(g) = 0 \quad \forall g \in \text{Goal}, \quad f(c) = +\infty \quad \forall c \notin \text{Goal} \quad (11)$$

The relaxation step for each cell c and passable direction d with neighbor $n = c + \Delta(d)$ is:

$$f(n) \leftarrow \min(f(n), f(c) + \text{cost}(d)) \quad (12)$$

This recurrence is applied until the priority queue is exhausted, at which point $f(c)$ contains the exact shortest-path distance from c to the nearest goal cell.

B. Computational Properties

Using a binary min-heap, the algorithm achieves:

- **Time Complexity:** $O(N^2 \log N^2) = O(N^2 \log N)$. For $N = 16$, this is approximately 2560 operations, essentially instantaneous on modern hardware.
- **Space Complexity:** $O(N^2)$, storing the distance value and heap entry for each cell.

A useful variant seeds the field at a single cell (typically the start), computing the distance-to-origin field:

$$f_{\text{start}}(s) = 0, \quad f_{\text{start}}(c) = +\infty \quad \forall c \neq s \quad (13)$$

This variant is used by the Trémaux explorer to navigate back to known territory when backtracking.

IV. A* PATH PLANNING

A. Algorithm Formulation

A* search finds the optimal path on the pessimistic (known-safe) map. The open-set priority for a cell c is:

$$F(c) = g(c) + h(c) \quad (14)$$

where $g(c)$ is the actual accumulated cost from the start to c (sum of movement costs along the path), and $h(c) = f(c)$ is the flood-fill distance serving as the heuristic estimate of remaining cost.

B. Admissibility and Optimality

Theorem IV.1 (Admissibility). *The heuristic $h(c) = f(c)$ is admissible, i.e., $h(c) \leq h^*(c)$ for all $c \in \mathcal{C}$, where $h^*(c)$ is the true optimal cost on the pessimistic map.*

Proof. The value $f(c)$ is computed by Dijkstra's algorithm on the fully-connected graph where every possible edge exists (optimistic passability). The pessimistic map is a subgraph of this fully-connected graph, as some edges are blocked by known walls. Therefore, any path on the pessimistic graph is also a path on the fully-connected graph, but not vice versa. The minimum cost on a supergraph cannot exceed the minimum cost on a subgraph, so $f(c) \leq h^*(c)$ for all c . $\square$

Theorem IV.2 (Consistency). *The heuristic $h(c) = f(c)$ is consistent (monotonic), i.e., $h(c) \leq \text{cost}(c \rightarrow n) + h(n)$ for all traversable edges (c, n) .*

Proof. This follows directly from the Dijkstra relaxation step. For any neighbor n reachable from c , the algorithm ensures

$f(c) \leq \text{cost}(d) + f(n)$, as otherwise $f(c)$ would have been updated to the smaller value. $\square$

Consistency is a stronger condition than admissibility; it guarantees that A* can be implemented without re-opening closed nodes, as the first time a node is expanded, the optimal path to it has been found.

C. Stale Node Detection

To avoid expensive decrease-key operations on the priority queue, we employ a lazy deletion strategy. When a node (F, c) is popped from the heap, we check:

$$F - h(c) > g(c) + \varepsilon \quad (15)$$

where $\varepsilon = 10^{-4}$ accounts for floating-point errors. If this condition holds, the entry is stale—a cheaper path to c was found after this entry was inserted—and the node is discarded.

D. Path Reconstruction

During expansion, the predecessor of each cell on the optimal path is recorded:

$$\text{parent}[n] = c \quad \text{when } g(n) \text{ is updated via } c \quad (16)$$

The path is reconstructed by backtracking from the goal to the start, then reversing the sequence.

E. Complexity Analysis

- **Worst Case:** $O(N^2 \log N^2)$, when all cells are expanded (equivalent to Dijkstra).
- **Typical Case:** $O(k \log k)$ where $k \ll N^2$, as the tight heuristic focuses the search near the optimal path.
- **Space:** $O(N^2)$ for the g -cost and parent arrays.

V. PATH POST-PROCESSING

A. Run-Length Encoding

The raw A* output is a cell sequence $c_0 \rightarrow c_1 \rightarrow \dots \rightarrow c_n$. Consecutive steps in the same direction are compressed into move primitives:

$$\text{Move}_i : (\text{direction } d_i, \text{count } k_i, \text{cost } s_i = k_i \cdot \text{cost}(d_i)) \quad (17)$$

The merge rule is: if $d_i = d_{i-1}$, then $k_{i-1} \leftarrow k_{i-1} + 1$ and $s_{i-1} \leftarrow s_{i-1} + \text{cost}(d_i)$. This reduces the representation from per-cell to per-corridor, enabling efficient geometric smoothing.

B. String-Pulling Smoothing

A greedy visibility-based algorithm removes redundant waypoints. Starting with the sequence of turn points, the algorithm iteratively checks whether the last retained point has line-of-sight (LoS) to a point two steps ahead:

```

out = [c_0]
for i = 1 to n-1:
    if LoS(out.last, c_{i+1}):
        skip c_i

```

```

else:
    append c_i
append c_n

```

This process repeats until no further points can be removed, requiring at most $O(n)$ passes and $O(n^2)$ total time.

The LoS condition is conservative:

$$\text{LoS}(a, b) = \text{TRUE} \iff \begin{cases} \Delta x(a, b) = 0 \text{ or} \\ \Delta y(a, b) = 0 \text{ or} \\ |\Delta x| = |\Delta y| \end{cases} \quad (18)$$

and all intermediate cells along the implied straight line are passable. Only axis-aligned or exact-diagonal lines are considered; arbitrary segments return false to guarantee collision-free paths.

VI. TRÉMAUX EXPLORER

A. Algorithm Overview

The Trémaux algorithm, dating to the 19th century, guarantees complete exploration of any finite maze. Our implementation enhances the classic algorithm with heuristic guidance and information-theoretic scoring.

B. Candidate Scoring

For each passable (optimistic) cardinal neighbor n of the current cell c , we compute a lexicographic score:

$$\text{score}(n) = (\text{visits}(n), f(n), -\text{gain}(n)) \quad (19)$$

where:

- $\text{visits}(n)$ is the visit count of cell n ; unvisited cells (0) are preferred.
- $f(n)$ is the flood-fill distance to the goal; smaller values (closer to goal) are preferred.
- $\text{gain}(n)$ is the information gain, defined below; higher values are preferred (minimizing $-\text{gain}$ achieves this).

C. Information Gain Metric

The information gain of a cell quantifies the amount of unknown wall state that would be revealed by visiting:

$$\text{gain}(c) = \sum_{d \in \{N, E, S, W\}} \text{canMove}(c, d) \cdot |\{w : K(c + \Delta(d), w) = 0\}| \quad (20)$$

where $\text{canMove}(c, d)$ indicates whether the robot can physically move in direction d (the passage is known to be open). This metric prioritizes cells that are junctions into unexplored territory, maximizing the rate of map discovery.

D. Backtracking Mechanism

When all optimistic paths from the current cell lead to visited territory, the robot has exhausted its local frontier. The backtracking procedure is:

- 1) Pop the backtrack stack until a cell b is found with at least one unvisited, reachable neighbor.
- 2) Plan a path from the current cell to b using A* on the pessimistic map.
- 3) Navigate to b and resume exploration.

This approach efficiently retraces steps through known-safe corridors rather than blindly reversing the exploration path.

E. Termination Guarantee

Theorem VI.1 (Trémaux Completeness). *The exploration procedure terminates in at most $2|\mathcal{E}|$ cell transitions, where $\mathcal{E}$ is the set of edges in the maze graph, and upon termination, all reachable cells have been visited at least once.*

Proof. The classic Trémaux property states that each edge is traversed at most twice (once in each direction). Our scoring function preserves this property by strictly preferring unvisited cells and only traversing previously-used edges during backtracking. In a graph with $|\mathcal{E}| \leq 4N^2$ edges (each cell has at most 4 cardinal connections), the upper bound is $2 \cdot 4N^2 = 8N^2 = 2048$ steps for $N = 16$.

The frontier of unvisited-but-reachable cells shrinks monotonically because: (1) the robot always moves to an unvisited cell when one is reachable, and (2) when no such cell exists, the algorithm backtracks until it finds one. Since the graph is finite, the process must terminate with all reachable cells visited. $\square$

VII. CLOTHOID-BASED TRAJECTORY GENERATION

A. Theoretical Foundation

A path consisting of straight lines and circular arcs has discontinuous curvature, causing instantaneous jumps in lateral acceleration that are physically unrealizable. The clothoid (Cornu spiral) solves this by providing C^2 continuity.

The clothoid is defined by linear curvature variation with respect to arc length:

$$\kappa(s) = \kappa_0 + \frac{\kappa_1 - \kappa_0}{L} \cdot s, \quad s \in [0, L] \quad (21)$$

Equivalently, $\frac{d\kappa}{ds} = \text{constant}$, which is the defining property of the clothoid.

B. Curvature Integration

The heading angle at arc length s is obtained by integrating curvature:

$$\theta(s) = \theta_0 + \int_0^s \kappa(\sigma) d\sigma \quad (22)$$

Substituting the linear curvature profile:

$$\theta(s) = \theta_0 + \kappa_0 s + \frac{1}{2} \cdot \frac{\kappa_1 - \kappa_0}{L} \cdot s^2 \quad (23)$$

The position is obtained via the Fresnel integrals:

$$\begin{aligned} x(s) &= x_0 + \int_0^s \cos \theta(\sigma) d\sigma \\ y(s) &= y_0 + \int_0^s \sin \theta(\sigma) d\sigma \end{aligned} \quad (24)$$

These integrals have no closed-form solution and are approximated numerically using a rectangular sum with $N = 10$ samples:

$$\begin{aligned} x(s) &\approx x_0 + \sum_{i=0}^9 \cos \left(\theta \left((i + 0.5) \cdot \frac{s}{10} \right) \right) \cdot \frac{s}{10} \\ y(s) &\approx y_0 + \sum_{i=0}^9 \sin \left(\theta \left((i + 0.5) \cdot \frac{s}{10} \right) \right) \cdot \frac{s}{10} \end{aligned} \quad (25)$$

C. Composite Turn Construction

A turn is composed of three segments stitched together with C^2 continuity:

- 1) **Entry Clothoid:** Curvature ramps from the initial value κ_{prev} to the turn curvature κ_{turn} over length L_c .
- 2) **Circular Arc:** Curvature remains constant at κ_{turn} for the main turn body.
- 3) **Exit Clothoid:** Curvature ramps from κ_{turn} to zero over length L_c .

The turn geometry is determined by:

- $\Delta\theta = \theta_{\text{exit}} - \theta_{\text{entry}}$ (normalized to $[-\pi, \pi]$)
- $L_{\text{seg}} = \|\mathbf{p}_{\text{exit}} - \mathbf{p}_{\text{entry}}\|_2$
- $R = \frac{L_{\text{seg}}}{2 \sin(|\Delta\theta|/2)}$ (chord-radius relation)
- $\kappa_{\text{turn}} = \text{sign}(\Delta\theta)/R$
- $L_c = \min \left(\sqrt{|\kappa_{\text{turn}}|/5}, 0.4 \cdot L_{\text{seg}} \right)$

The clothoid ramp length is bounded by a bandwidth rule-of-thumb and capped at 40% of the segment length to leave sufficient room for the circular arc.

D. C^2 Continuity Verification

We verify the continuity conditions at the transition points between segments. Let $s_1 = L_c$ be the arc length at the clothoid-arc transition, and $s_2 = L_c + L_{\text{arc}}$ be the arc length at the arc-clothoid transition.

C^2 Continuity. At the entry clothoid-circular arc junction ($s = L_c$):

- Position and tangent (C^1): The Fresnel integration guarantees continuous position and heading by construction, as $\theta(s)$ is a continuous integral of the continuous $\kappa(s)$.
- Curvature (C^2): From the linear ramp definition, $\kappa(L_c) = \kappa_0 + \frac{\kappa_1 - \kappa_0}{L_c} \cdot L_c = \kappa_1 = \kappa_{\text{turn}}$, which matches the arc curvature exactly.

At the circular arc-exit clothoid junction ($s = s_2$), the exit clothoid starts with κ_{turn} and ramps to zero, ensuring curvature continuity. The curvature function is continuous everywhere, with only its derivative changing at segment boundaries. $\square$

VIII. JERK-LIMITED VELOCITY PROFILING

A. Physical Constraints

The velocity profile must satisfy three simultaneous constraints:

- 1) **Centripetal Constraint:** $v \leq \sqrt{a_{\text{lat}}/|\kappa|}$
- 2) **Longitudinal Jerk Constraint:** $|da/dt| \leq J_{\text{max}}$
- 3) **Braking Constraint:** $v \leq v_{\text{brake}}(s)$ such that stopping at the goal is guaranteed

The physical parameters are summarized in Table I.

TABLE I
PHYSICAL PARAMETER VALUES

Symbol	Value	Meaning
a_{max}	9.0 m/s ²	Max longitudinal acceleration
a_{brake}	10.0 m/s ²	Max braking deceleration
a_{lat}	8.0 m/s ²	Max lateral (centripetal)
a_{total}	12.0 m/s ²	Friction circle limit
J_{max}	80.0 m/s ³	Max jerk
v_{max}	5.0 m/s	Speed-run top speed

B. Two-Pass Algorithm

Forward Pass (acceleration-limited):

For $i = 1, 2, \dots, n - 1$:

$$\begin{aligned} ds_i &= s_i - s_{i-1} \\ a_{\text{reach}} &= \min(a_{i-1} + J_{\text{max}} \cdot ds_i, a_{\text{max}}) \\ v_{\text{acc}} &= \sqrt{v_{i-1}^2 + 2 \cdot a_{\text{reach}} \cdot ds_i} \\ v_{\text{curv}} &= \sqrt{a_{\text{lat}}/|\kappa_i|} \\ v_i &= \min(v_i, v_{\text{acc}}, v_{\text{curv}}) \\ a_i &= (v_i^2 - v_{i-1}^2)/(2 \cdot ds_i) \end{aligned} \quad (26)$$

Backward Pass (braking-limited):

$$\begin{aligned} v_n &= 0 \quad (\text{stop at goal}) \\ \text{For } i &= n - 1 \text{ down to } 0: \\ ds_i &= s_{i+1} - s_i \\ a_{\text{reach}} &= \min(a_{i+1} + J_{\text{max}} \cdot ds_i, a_{\text{brake}}) \\ v_{\text{brake}} &= \sqrt{v_{i+1}^2 + 2 \cdot a_{\text{reach}} \cdot ds_i} \\ v_i &= \min(v_i, v_{\text{brake}}) \end{aligned} \quad (27)$$

The backward pass is crucial: it propagates the zero-velocity boundary condition at the goal backward along the trajectory, ensuring that the forward-computed velocity profile is consistent with the requirement to stop at the end.

C. Time Estimation

Given the final velocity profile, the traversal time is estimated using trapezoidal integration:

$$T = \sum_i \frac{ds_i}{(v_i + v_{i-1})/2} \quad (28)$$

IX. KALMAN FILTER STATE ESTIMATION

A. State and Covariance Model

The robot maintains a 3-state estimate:

$$\mathbf{x} = [x, y, \theta]^T \quad (29)$$

representing position in meters and heading in radians. The covariance is maintained as a diagonal matrix:

$$\mathbf{P} = \text{diag}(P_{xx}, P_{yy}, P_{\theta\theta}) \quad (30)$$

The diagonal assumption simplifies computation while capturing the dominant error modes: longitudinal/lateral drift and heading uncertainty. The initial covariance is:

$$\mathbf{P}_0 = \text{diag}(0.01, 0.01, 0.001) \quad (31)$$

B. Predict Step

Given encoder measurements of distance traveled Δs and heading change $\Delta\theta$, the state is propagated using a midpoint heading model:

$$\begin{aligned} x^- &\leftarrow x + \Delta s \cdot \cos(\theta + \Delta\theta/2) \\ y^- &\leftarrow y + \Delta s \cdot \sin(\theta + \Delta\theta/2) \\ \theta^- &\leftarrow \theta + \Delta\theta \end{aligned} \quad (32)$$

The midpoint heading model provides second-order accuracy for the nonlinear motion integration, reducing linearization errors compared to a simple Euler step.

The covariance is updated with motion-proportional process noise:

$$\begin{aligned} P_{xx}^- &\leftarrow P_{xx} + Q_{xy} + \cos^2 \theta \cdot \Delta s \cdot 0.01 \\ P_{yy}^- &\leftarrow P_{yy} + Q_{xy} + \sin^2 \theta \cdot \Delta s \cdot 0.01 \\ P_{\theta\theta}^- &\leftarrow P_{\theta\theta} + Q_{\theta} + |\Delta\theta| \cdot 0.1 \end{aligned} \quad (33)$$

with noise parameters $Q_{xy} = 10^{-5} \text{ m}^2$ and $Q_{\theta} = 10^{-4} \text{ rad}^2$. The motion-proportional terms reflect the physical reality that longer motions accumulate more uncertainty.

C. Update Step

When the robot drives parallel to a known wall, its side-facing sensors provide a distance measurement z . The innovation is:

$$\nu = z - \hat{z} \quad (34)$$

where $\hat{z}$ is the predicted distance based on the current state estimate and the known wall position. For the relevant axis $a \in \{x, y\}$:

$$\begin{aligned} S &= P_{aa} + R_{\text{wall}} \\ K &= P_{aa}/S \\ x_a &\leftarrow x_a + K \cdot \nu \\ P_{aa} &\leftarrow P_{aa} \cdot (1 - K) \end{aligned} \quad (35)$$

with measurement noise $R_{\text{wall}} = 0.003 \text{ m}^2$.

The Kalman gain $K \in [0, 1]$ optimally weights the prediction and measurement based on their relative uncertainties. When the prediction is confident (P_{aa} small), the measurement has little influence; when the prediction is uncertain (P_{aa} large), the measurement provides strong correction.

X. SYSTEM INTEGRATION AND COMPLEXITY

A. Computational Complexity Summary

Table II presents the computational complexity of each component.

TABLE II
COMPONENT COMPLEXITY ANALYSIS

Component	Time	Space	Guarantee
Flood-fill	$O(N^2 \log N)$	$O(N^2)$	Exact distances
A* Planner	$O(N^2 \log N)$	$O(N^2)$	Optimal path
Path Smoother	$O(n^2)$	$O(n)$	Locally minimal
Trémaux	$O(N^2)$	$O(N^2)$	Full coverage
Clothoid Gen.	$O(m \cdot T)$	$O(m \cdot T)$	C ² continuity
Velocity Prof.	$O(T)$	$O(T)$	Jerk-limited
Kalman Filter	$O(1)/\text{step}$	$O(1)$	MMSE estimate

Here $N = 16$ is the grid size, $n \leq 256$ is the path length, m is the number of moves, and T is the number of trajectory samples.

B. System Invariants

The correctness of the overall system is guaranteed by maintaining the following invariants throughout operation:

- 1) **Heuristic Admissibility:** $h(c) = f(c) \leq h^*(c)$ for all c , ensuring A* optimality.
- 2) **Wall Symmetry:** $W(c, w) = W(\text{adj}(c, w), \text{opp}(w))$, ensuring physical consistency.
- 3) **Trémaux Coverage:** $\text{visitCount}(c) \leq 2$ for all reachable cells, ensuring termination.
- 4) **Velocity Limits:** $v_i \leq \sqrt{a_{\text{lat}}/|\kappa_i|}$ and $v_0 = v_n = 0$, ensuring physical feasibility.
- 5) **Clothoid C²:** $\kappa(s)$ continuous everywhere; $d\kappa/ds$ piecewise constant.
- 6) **Kalman Consistency:** $\mathbb{E}[(\mathbf{x} - \hat{\mathbf{x}})(\mathbf{x} - \hat{\mathbf{x}})^T] \leq \mathbf{P}$ by design.

XI. CONCLUSION

We have presented a complete, mathematically rigorous navigation architecture for autonomous maze solving. Each component is grounded in well-established theory: Dijkstra's algorithm provides exact distance computations, A* guarantees optimal path planning through admissible heuristics, the Trémaux algorithm ensures complete exploration, clothoid curves provide C² continuous trajectories, and the Kalman filter optimally fuses sensor measurements. The integration of these components, with careful attention to the interfaces between them, yields a system that is provably correct, computationally efficient, and physically realizable.

The architecture's separation of concerns—mapping, planning, exploration, trajectory generation, and state estimation—enables independent verification of each component

while maintaining system-wide guarantees through the invariant properties identified in this paper. Future work includes extension to dynamic environments, integration of learned heuristics, and adaptation to three-dimensional navigation problems.

REFERENCES

- [1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [2] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [3] M. Trémaux, "Solution of the Labyrinth Problem," *Proceedings of the French Academy of Sciences*, 1882.
- [4] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [5] A. Kelly and B. Nagy, "Reactive Nonholonomic Trajectory Generation via Parametric Optimal Control," *International Journal of Robotics Research*, vol. 22, no. 7, pp. 583–601, 2003.
- [6] J. C. Latombe, *Robot Motion Planning*. Kluwer Academic Publishers, 1991.
- [7] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2007.
- [8] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.